In the last section, we implemented distributed advantage actor-critic, except that we trained in Monte Carlo mode—we ran a full episode before updating the model parameters. While that makes sense for a simple game like CartPole, usually we want to be able to make more frequent updates. We briefly touched on *N*-step learning before, but to reiterate, it means we simply calculate our loss and update the parameters after *N* steps, where *N* is whatever we choose it to be. If *N* is 1, this is fully online learning; if *N* is very large, it will be Monte Carlo again. The sweet spot is somewhere in between.

With Monte Carlo full-episode learning, we don't take advantage of bootstrapping, since there's nothing to bootstrap. We do bootstrap in online learning, as we did with DQN, but with 1-step learning the bootstrap may introduce a lot of bias. This bias may be harmless if it pushes our parameters in the right direction, but in some cases the bias can be so off that we never move in the right direction.

This is why *N*-step learning is usually better than 1-step (online) learning—the target value for the critic is more accurate, so the critic's training will be more stable and will be able to produce less biased state values. With bootstrapping, we're making a prediction from a prediction, so the predictions will be better if you're able to collect more data before making them. And we like bootstrapping because it improves sample efficiency; you don't need to see as much data (e.g., frames in a game) before updating the parameters in the right direction.

Let's modify our code to do *N*-step learning. The only function we need to modify is `run_episode`. We need to change it to run for only *N* steps rather than wait for the episode to finish. If the episode finishes before *N* steps, the last return value will be set to 0 (since there is no next state when the game is over) as it was in the Monte Carlo case. However, if the episode hasn't finished by *N* steps, we'll use the last state value as our prediction for what the return would have been had we kept playing—that's where the bootstrapping happens. Without bootstrapping, the critic is just trying to predict the future returns from a state, and it gets the actual returns as training data. With bootstrapping, it is still trying to predict future returns,